

Toy: data records

- ❖ why not encrypt data in constant stream as we write it to TCP?
 - where would we put the MAC? If at end, no message integrity until all data processed.
 - e.g., with instant messaging, how can we do integrity check over all bytes sent before displaying?
- ❖ instead, break stream in series of records
 - each record carries a MAC
 - receiver can act on each record as it arrives
- ❖ issue: in record, receiver needs to distinguish MAC from data
 - want to use variable-length records



Toy: sequence numbers

- ❖ *problem*: attacker can capture and replay record or re-order records
- ❖ *solution*: put sequence number into MAC:
 - $MAC = MAC(M_x, \text{sequence}||\text{data})$
 - note: no sequence number field
- ❖ *problem*: attacker could replay all records
- ❖ *solution*: use nonce

Toy: control information

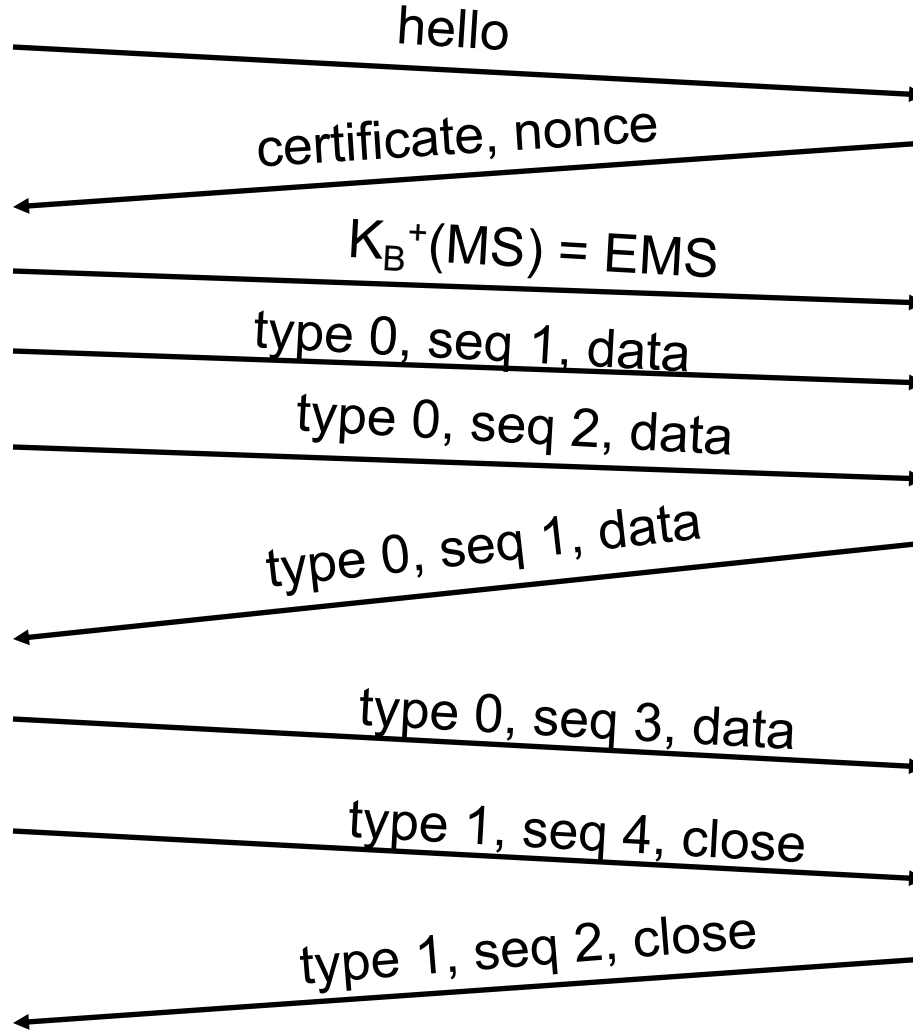
- ❖ *problem*: truncation attack:
 - attacker forges TCP connection close segment
 - one or both sides thinks there is less data than there actually is.
- ❖ *solution*: record types, with one type for closure
 - type 0 for data; type 1 for closure
- ❖ $MAC = MAC(M_x, \text{sequence}||\text{type}||\text{data})$



Toy SSL: summary



encrypted



bob.com

Toy SSL isn't complete

- ❖ how long are fields?
- ❖ which encryption protocols?
- ❖ want negotiation?
 - allow client and server to support different encryption algorithms
 - allow client and server to choose together specific algorithm before data transfer

SSL cipher suite

❖ cipher suite

- public-key algorithm
- symmetric encryption algorithm
- MAC algorithm

❖ SSL supports several cipher suites

❖ negotiation: client, server agree on cipher suite

- client offers choice
- server picks one

common SSL symmetric ciphers

- DES – Data Encryption Standard: block
- 3DES – Triple strength: block
- RC2 – Rivest Cipher 2: block
- RC4 – Rivest Cipher 4: stream

SSL Public key encryption

- RSA

Real SSL: handshake (I)

Purpose

1. server authentication
2. negotiation: agree on crypto algorithms
3. establish keys
4. client authentication (optional)

Real SSL: handshake (2)

1. client sends list of algorithms it supports, along with client nonce
2. server chooses algorithms from list; sends back: choice + certificate + server nonce
3. client verifies certificate, extracts server's public key, generates pre_master_secret, encrypts with server's public key, sends to server
4. client and server independently compute encryption and MAC keys from pre_master_secret and nonces
5. client sends a MAC of all the handshake messages
6. server sends a MAC of all the handshake messages

Real SSL: handshaking (3)

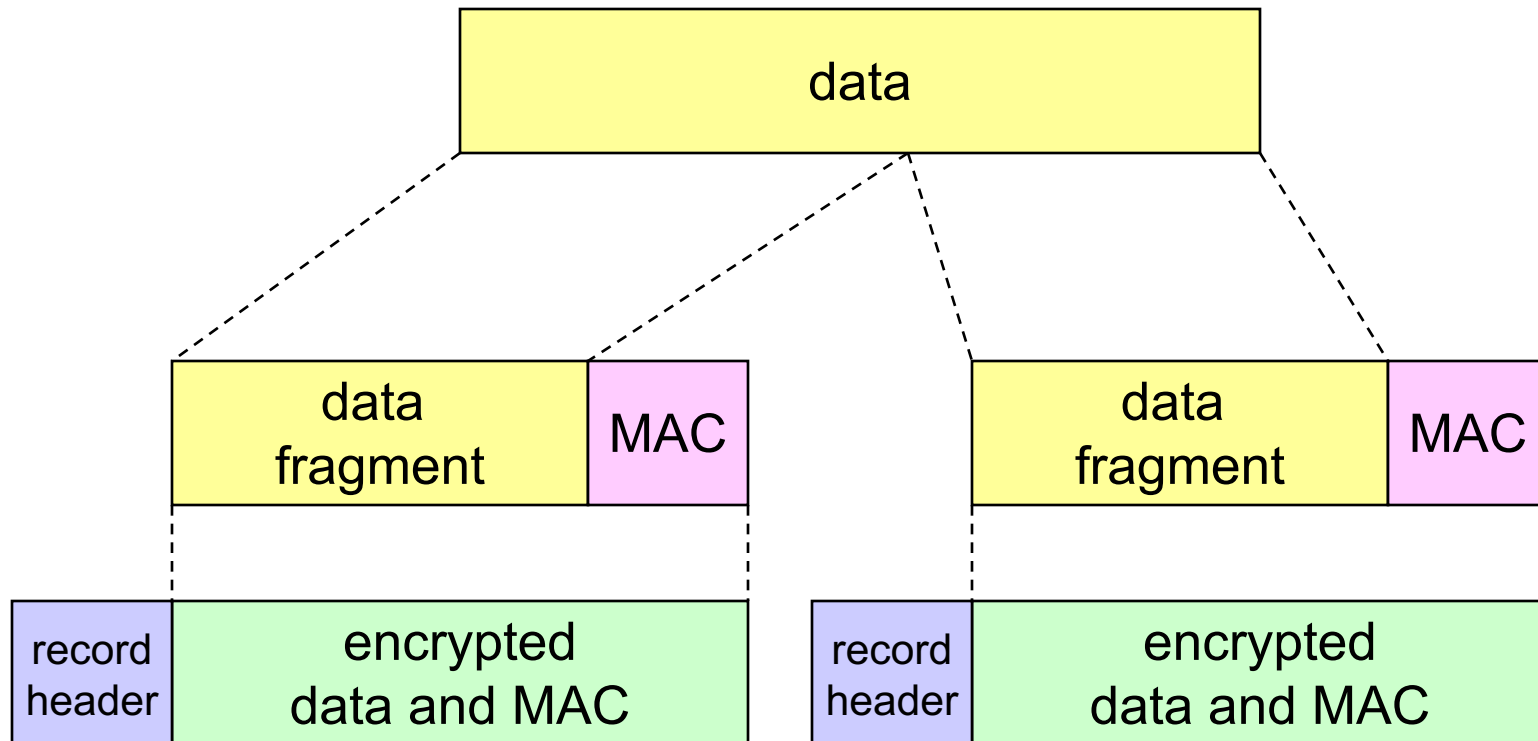
last 2 steps protect handshake from tampering

- ❖ client typically offers range of algorithms, some strong, some weak
- ❖ man-in-the middle could delete stronger algorithms from list
- ❖ last 2 steps prevent this
 - last two messages are encrypted

Real SSL: handshaking (4)

- ❖ why two random nonces?
- ❖ suppose Trudy sniffs all messages between Alice & Bob
- ❖ next day, Trudy sets up TCP connection with Bob, sends exact same sequence of records
 - Bob (Amazon) thinks Alice made two separate orders for the same thing
 - solution: Bob sends different random nonce for each connection. This causes encryption keys to be different on the two days
 - Trudy's messages will fail Bob's integrity check

SSL record protocol

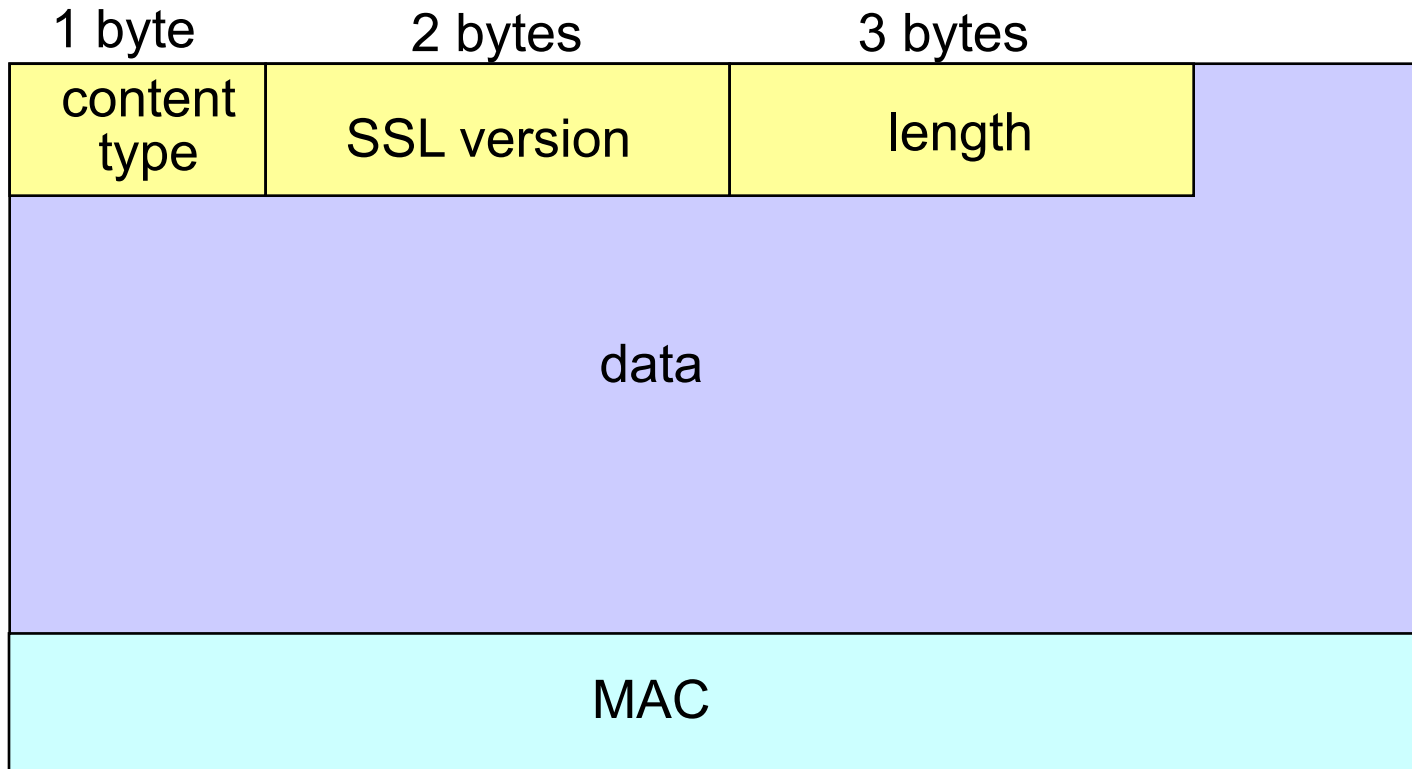


record header: content type; version; length

MAC: includes sequence number, MAC key M_x

fragment: each SSL fragment 2^{14} bytes (~16 Kbytes)

SSL record format

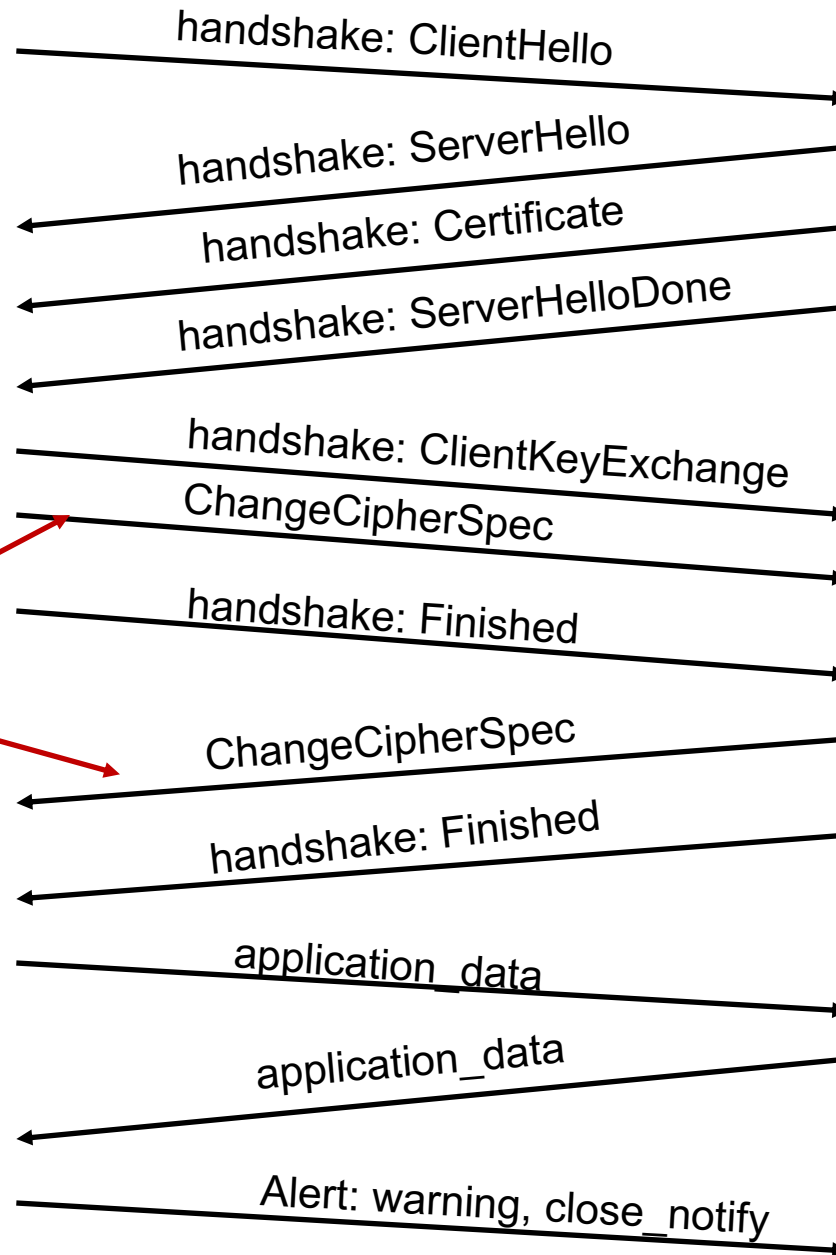


data and MAC encrypted (symmetric algorithm)

Real SSL connection

*everything
henceforth
is encrypted*

TCP FIN follows



Key derivation

- ❖ client nonce, server nonce, and pre-master secret input into pseudo random-number generator.
 - produces master secret
- ❖ master secret and new nonces input into another random-number generator: “key block”
 - because of resumption: TBD
- ❖ key block sliced and diced:
 - client MAC key
 - server MAC key
 - client encryption key
 - server encryption key
 - client initialization vector (IV)
 - server initialization vector (IV)

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

What is network-layer confidentiality ?

between two network entities:

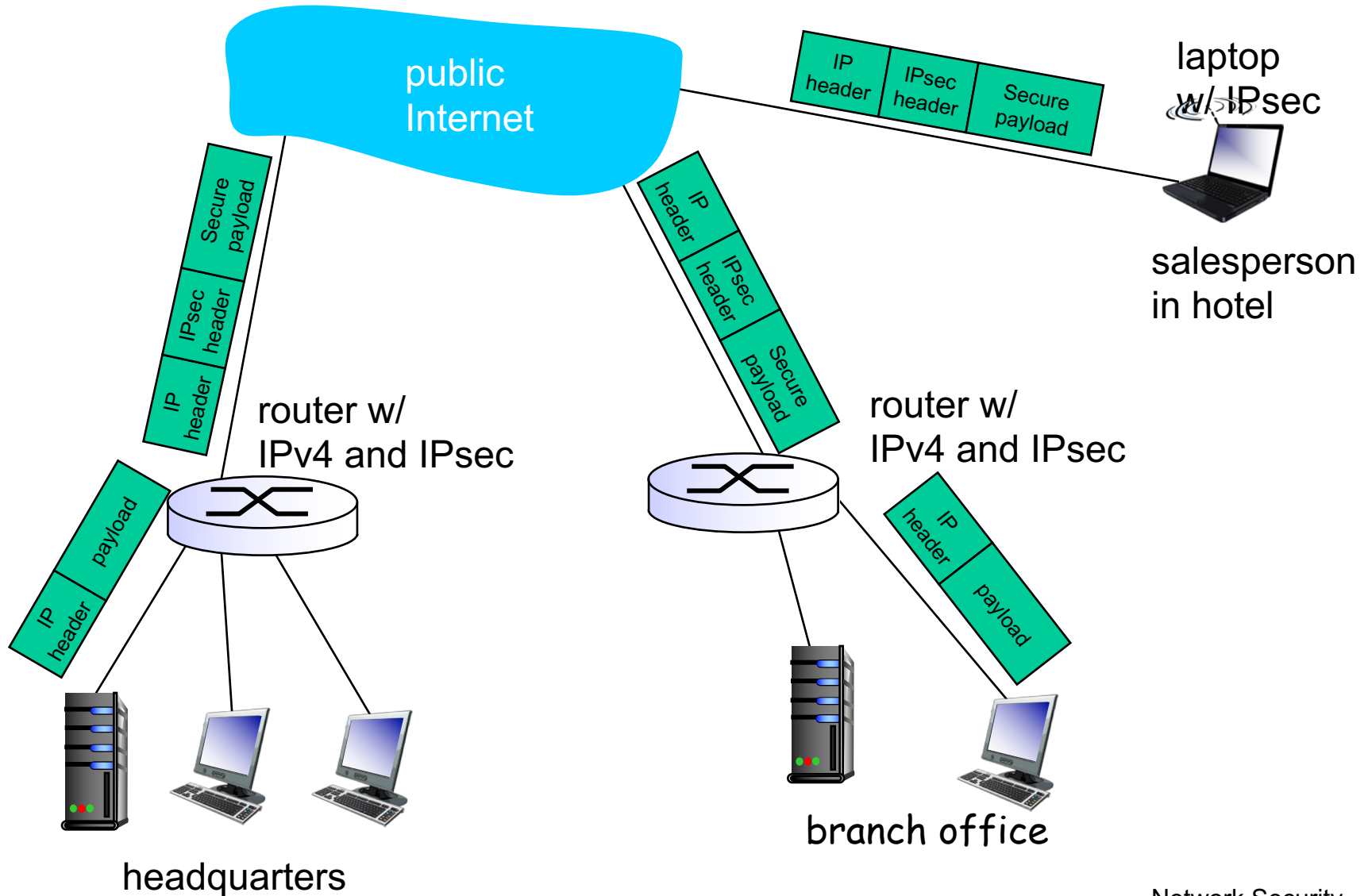
- ❖ sending entity encrypts datagram payload, payload could be:
 - TCP or UDP segment, ICMP message, OSPF message
- ❖ all data sent from one entity to other would be hidden:
 - web pages, e-mail, P2P file transfers, TCP SYN packets
 - ...
- ❖ “blanket coverage”

Virtual Private Networks (VPNs)

motivation:

- ❖ institutions often want private networks for security.
 - costly: separate routers, links, DNS infrastructure.
- ❖ VPN: institution's inter-office traffic is sent over public Internet instead
 - encrypted before entering public Internet
 - logically separate from other traffic

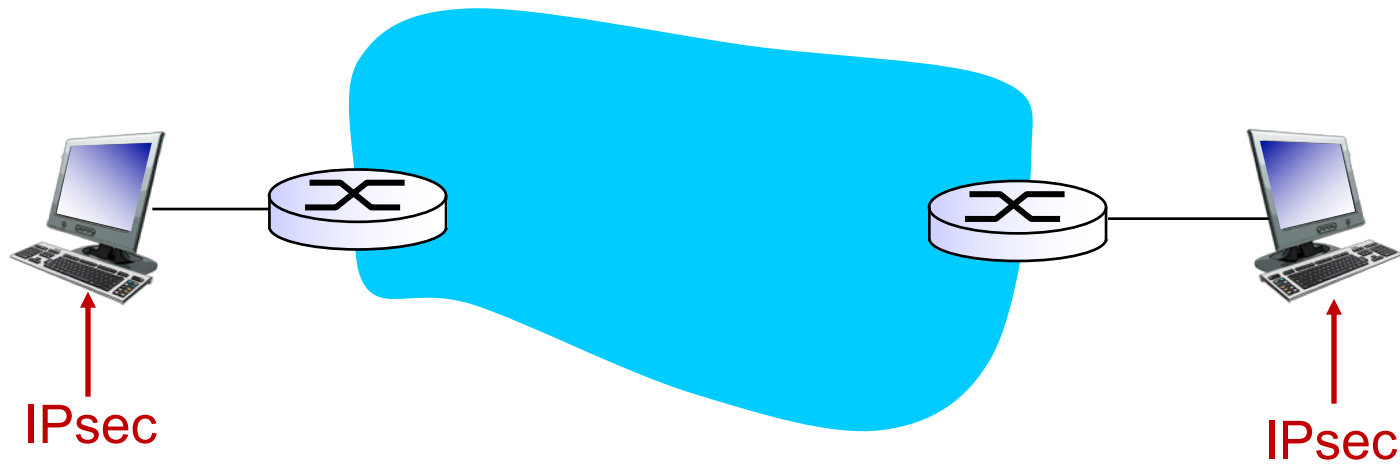
Virtual Private Networks (VPNs)



IPsec services

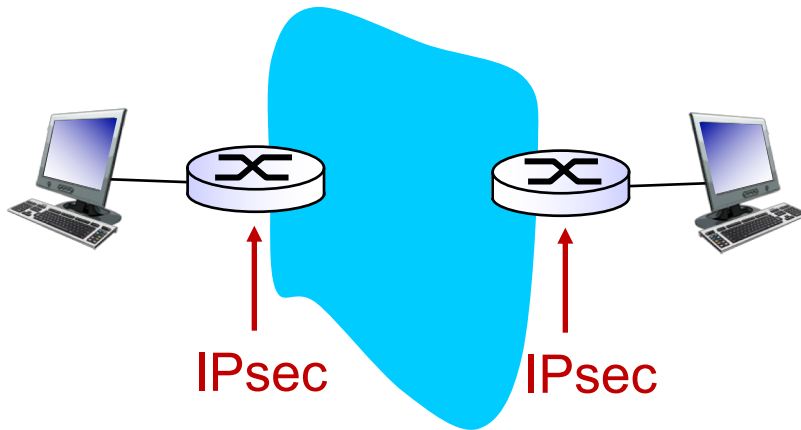
- ❖ data integrity
 - ❖ origin authentication
 - ❖ replay attack prevention
 - ❖ confidentiality
-
- ❖ two protocols providing different service models:
 - AH
 - ESP

IPsec transport mode

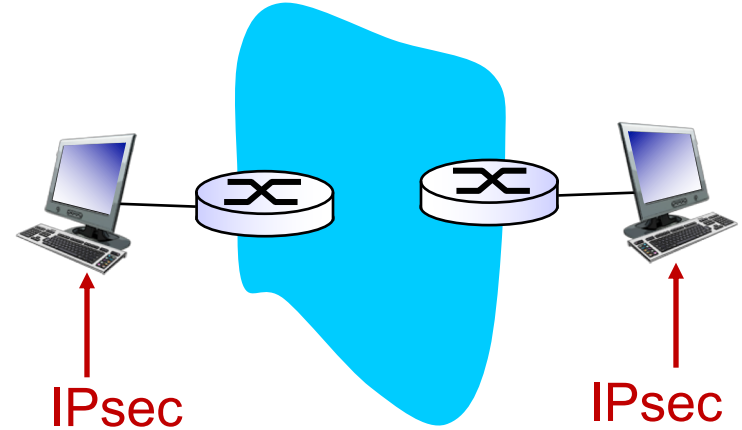


- ❖ IPsec datagram emitted and received by end-system
- ❖ protects upper level protocols

IPsec – tunneling mode



❖ edge routers IPsec-aware



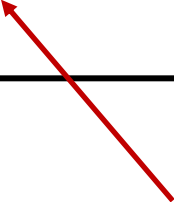
❖ hosts IPsec-aware

Two IPsec protocols

- ❖ Authentication Header (AH) protocol
 - provides source authentication & data integrity but *not* confidentiality
- ❖ Encapsulation Security Protocol (ESP)
 - provides source authentication, data integrity, *and* confidentiality
 - more widely used than AH

Four combinations are possible!

Host mode with AH	Host mode with ESP
Tunnel mode with AH	Tunnel mode with ESP

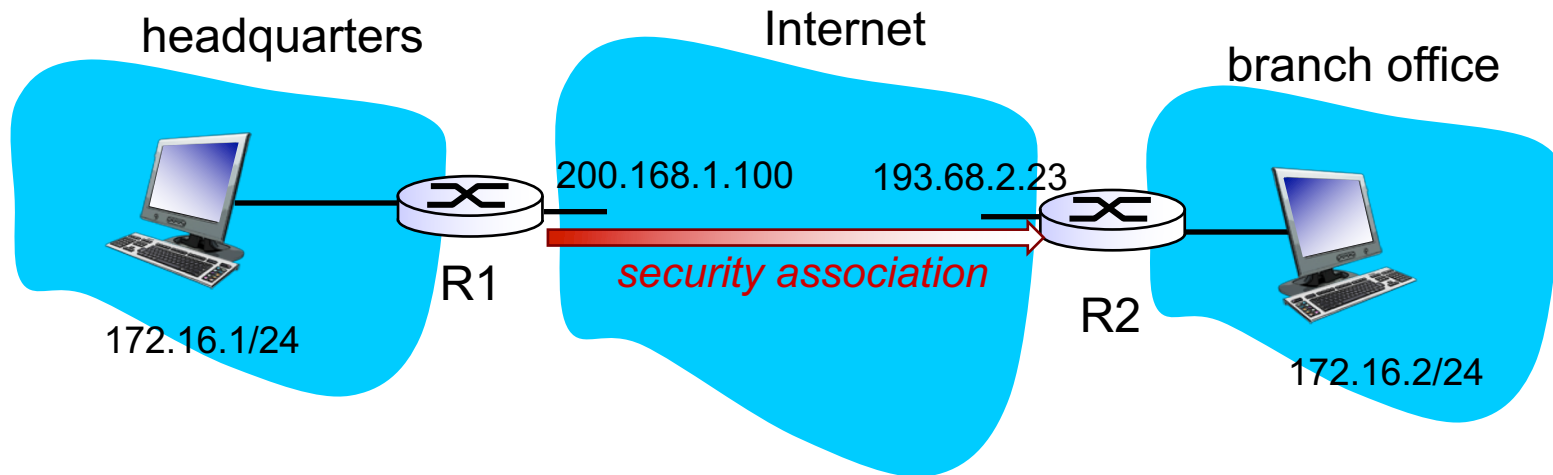


most common and
most important

Security associations (SAs)

- ❖ before sending data, “**security association (SA)**” established from sending to receiving entity
 - SAs are simplex: for only one direction
- ❖ ending, receiving entities maintain *state information* about SA
 - recall: TCP endpoints also maintain state info
 - IP is connectionless; IPsec is connection-oriented!
- ❖ how many SAs in VPN w/ headquarters, branch office, and n traveling salespeople?

Example SA from R1 to R2



R1 stores for SA:

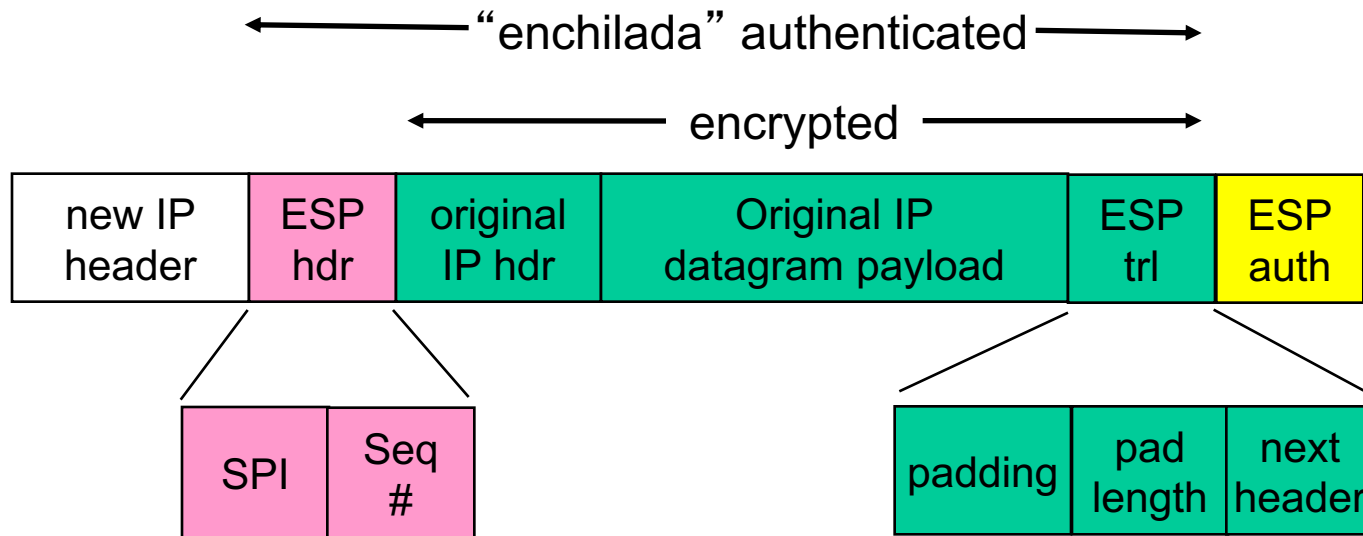
- ❖ 32-bit SA identifier: *Security Parameter Index (SPI)*
- ❖ origin SA interface (200.168.1.100)
- ❖ destination SA interface (193.68.2.23)
- ❖ type of encryption used (e.g., 3DES with CBC)
- ❖ encryption key
- ❖ type of integrity check used (e.g., HMAC with MD5)
- ❖ authentication key

Security Association Database (SAD)

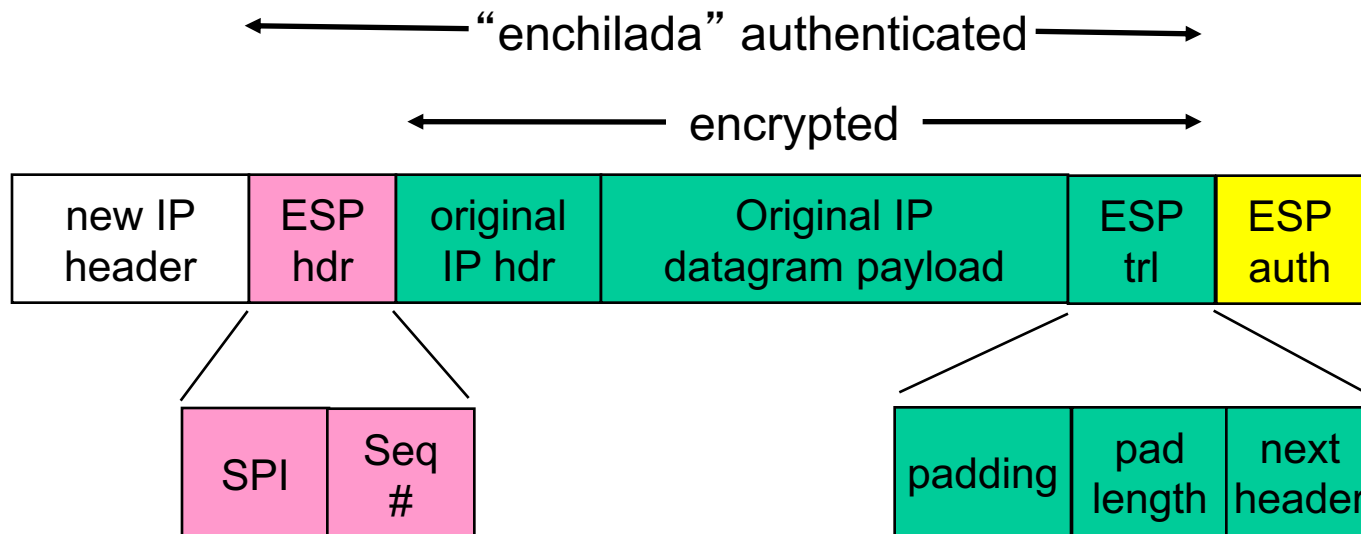
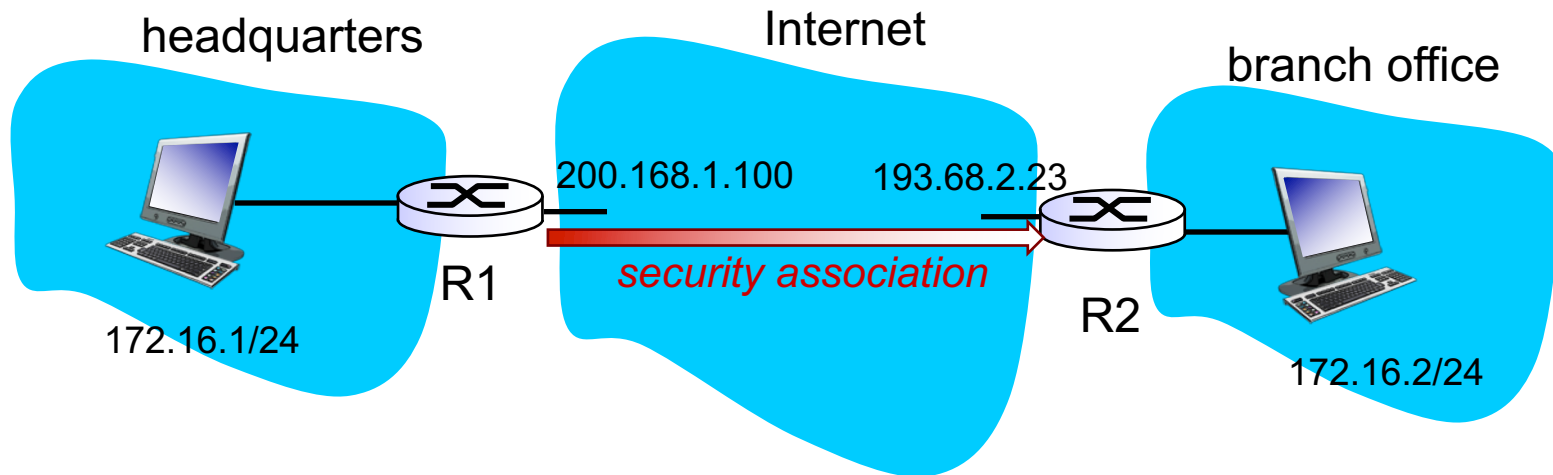
- ❖ endpoint holds SA state in *security association database (SAD)*, where it can locate them during processing.
- ❖ with n salespersons, $2 + 2n$ SAs in $R1$'s SAD
- ❖ when sending IPsec datagram, $R1$ accesses SAD to determine how to process datagram.
- ❖ when IPsec datagram arrives to $R2$, $R2$ examines SPI in IPsec datagram, indexes SAD with SPI, and processes datagram accordingly.

IPsec datagram

focus for now on tunnel mode with ESP



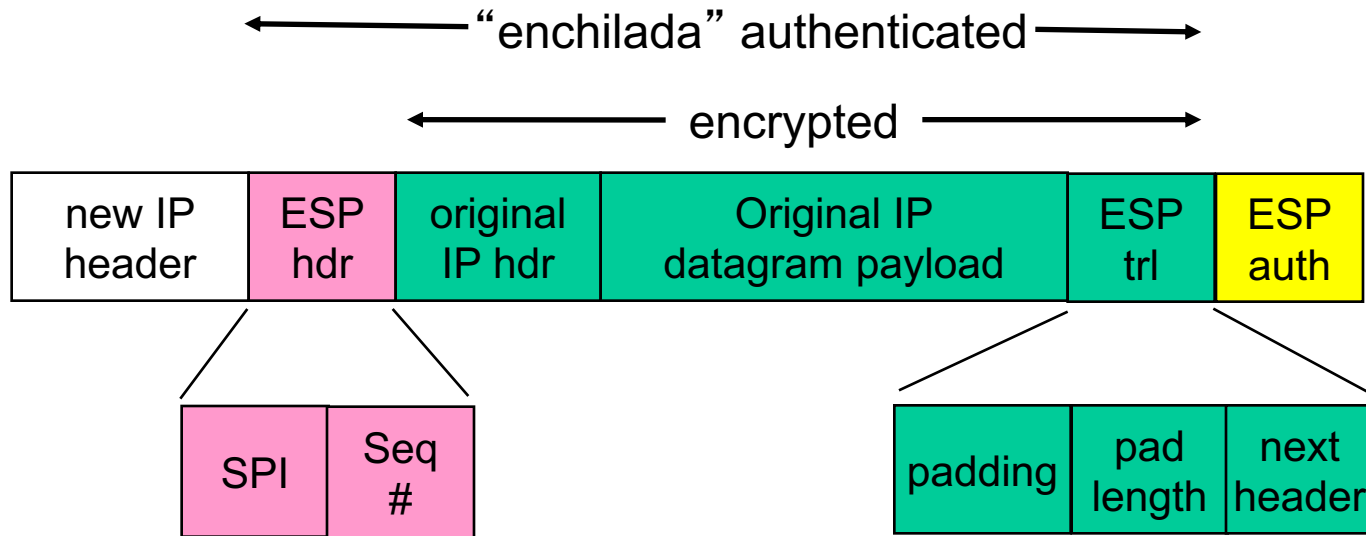
What happens?



RI: convert original datagram to IPsec datagram

- ❖ appends to back of original datagram (which includes original header fields!) an “ESP trailer” field.
- ❖ encrypts result using algorithm & key specified by SA.
- ❖ appends to front of this encrypted quantity the “ESP header, creating “enchilada”.
- ❖ creates authentication MAC over the *whole enchilada*, using algorithm and key specified in SA;
- ❖ appends MAC to back of enchilada, forming *payload*;
- ❖ creates brand new IP header, with all the classic IPv4 header fields, which it appends before payload.

Inside the enchilada:



- ❖ ESP trailer: Padding for block ciphers
- ❖ ESP header:
 - SPI, so receiving entity knows what to do
 - Sequence number, to thwart replay attacks
- ❖ MAC in ESP auth field is created with shared secret key

IPsec sequence numbers

- ❖ for new SA, sender initializes seq. # to 0
- ❖ each time datagram is sent on SA:
 - sender increments seq # counter
 - places value in seq # field
- ❖ goal:
 - prevent attacker from sniffing and replaying a packet
 - receipt of duplicate, authenticated IP packets may disrupt service
- ❖ method:
 - destination checks for duplicates
 - doesn't keep track of *all* received packets; instead uses a window

Security Policy Database (SPD)

- ❖ policy: For a given datagram, sending entity needs to know if it should use IPsec
- ❖ needs also to know which SA to use
 - may use: source and destination IP address; protocol number
- ❖ info in SPD indicates “what” to do with arriving datagram
- ❖ info in SAD indicates “how” to do it

Summary: IPsec services



- ❖ suppose Trudy sits somewhere between R1 and R2. she doesn't know the keys.
 - will Trudy be able to see original contents of datagram? How about source, dest IP address, transport protocol, application port?
 - flip bits without detection?
 - masquerade as R1 using R1's IP address?
 - replay a datagram?

IKE: Internet Key Exchange

- ❖ *previous examples*: manual establishment of IPsec SAs in IPsec endpoints:

Example SA

SPI: 12345

Source IP: 200.168.1.100

Dest IP: 193.68.2.23

Protocol: ESP

Encryption algorithm: 3DES-cbc

HMAC algorithm: MD5

Encryption key: 0x7aeaca...

HMAC key: 0xc0291f...

- ❖ manual keying is impractical for VPN with 100s of endpoints
- ❖ instead use *IPsec IKE (Internet Key Exchange)*

IKE: PSK and PKI

- ❖ authentication (prove who you are) with either
 - pre-shared secret (PSK) or
 - with PKI (public/private keys and certificates).
- ❖ PSK: both sides start with secret
 - run IKE to authenticate each other and to generate IPsec SAs (one in each direction), including encryption, authentication keys
- ❖ PKI: both sides start with public/private key pair, certificate
 - run IKE to authenticate each other, obtain IPsec SAs (one in each direction).
 - similar with handshake in SSL.

IKE phases

- ❖ IKE has two phases
 - *phase 1*: establish bi-directional IKE SA
 - note: IKE SA different from IPsec SA
 - aka ISAKMP security association
 - *phase 2*: ISAKMP is used to securely negotiate IPsec pair of SAs
- ❖ phase 1 has two modes: aggressive mode and main mode
 - aggressive mode uses fewer messages
 - main mode provides identity protection and is more flexible

IPsec summary

- ❖ IKE message exchange for algorithms, secret keys, SPI numbers
- ❖ either AH or ESP protocol (or both)
 - AH provides integrity, source authentication
 - ESP protocol (with AH) additionally provides encryption
- ❖ IPsec peers can be two end systems, two routers/firewalls, or a router/firewall and an end system

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

WEP design goals



❖ symmetric key crypto

- confidentiality
- end host authorization
- data integrity



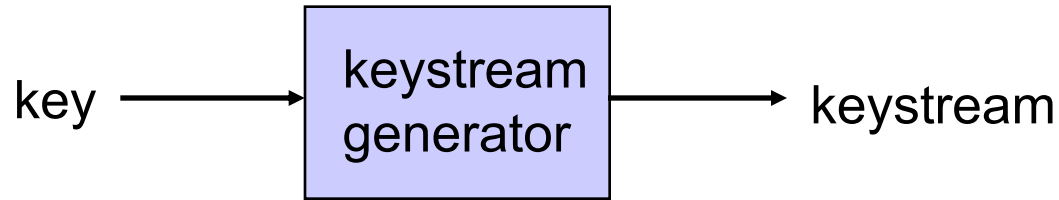
❖ self-synchronizing: each packet separately encrypted

- given encrypted packet and key, can decrypt; can continue to decrypt packets when preceding packet was lost (unlike Cipher Block Chaining (CBC) in block ciphers)

❖ Efficient

- implementable in hardware or software

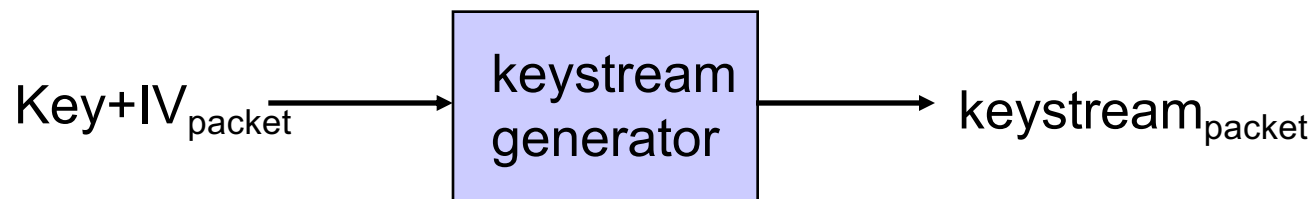
Review: symmetric stream ciphers



- ❖ *combine each byte of keystream with byte of plaintext to get ciphertext:*
 - $m(i)$ = ith unit of message
 - $ks(i)$ = ith unit of keystream
 - $c(i)$ = ith unit of ciphertext
 - $c(i) = ks(i) \oplus m(i)$ ($\oplus$ = exclusive or)
 - $m(i) = ks(i) \oplus c(i)$
- ❖ WEP uses RC4

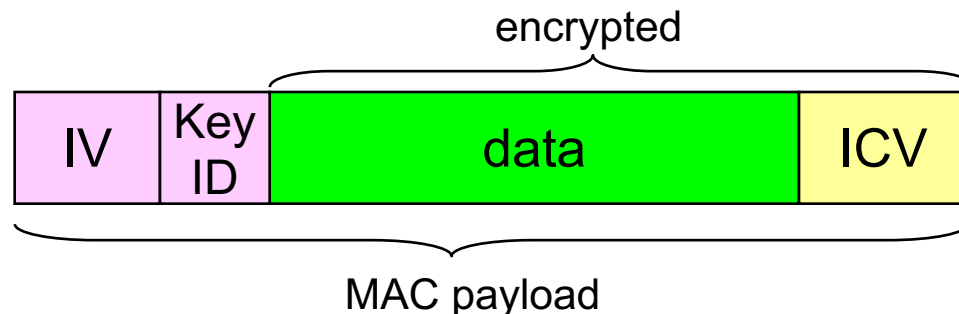
Stream cipher and packet independence

- ❖ recall design goal: each packet separately encrypted
- ❖ if for frame $n+1$, use keystream from where we left off for frame n , then each frame is not separately encrypted
 - need to know where we left off for packet n
- ❖ WEP approach: initialize keystream with key + new IV for each packet:

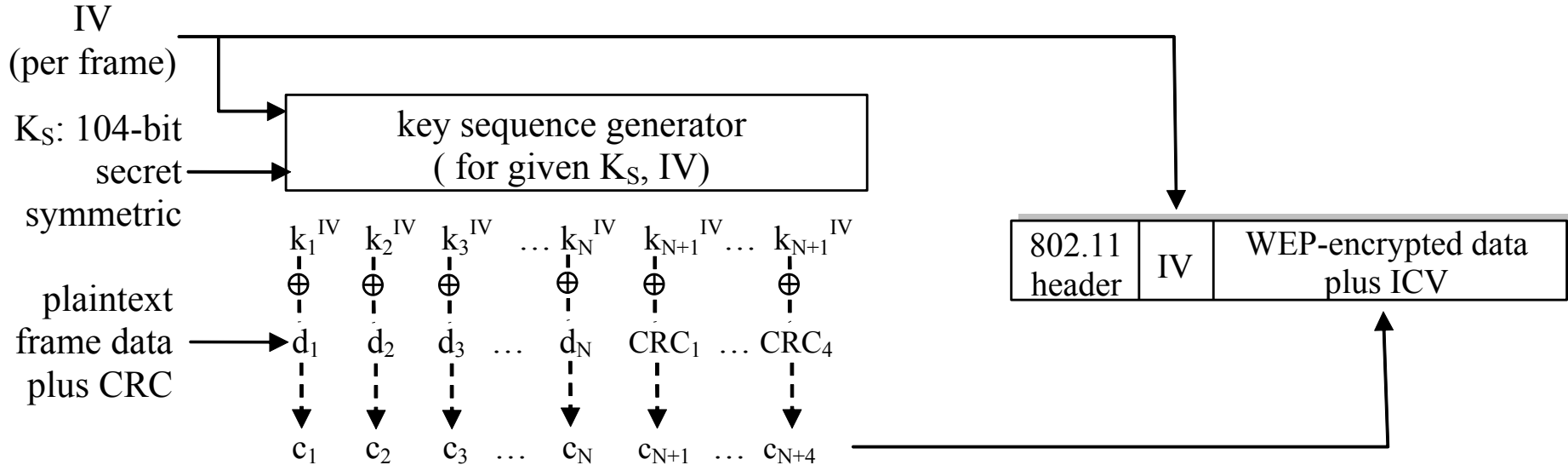


WEP encryption (I)

- ❖ sender calculates Integrity Check Value (ICV) over data
 - four-byte hash/CRC for data integrity
- ❖ each side has 104-bit shared key
- ❖ sender creates 24-bit initialization vector (IV), appends to key: gives 128-bit key
- ❖ sender also appends keyID (in 8-bit field)
- ❖ 128-bit key inputted into pseudo random number generator to get keystream
- ❖ data in frame + ICV is encrypted with RC4:
 - B\bytes of keystream are XORed with bytes of data & ICV
 - IV & keyID are appended to encrypted data to create payload
 - payload inserted into 802.11 frame

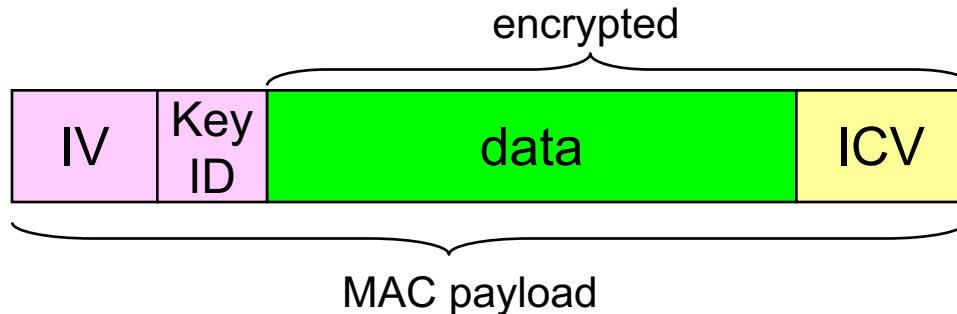


WEP encryption (2)



new IV for each frame

WEP decryption overview

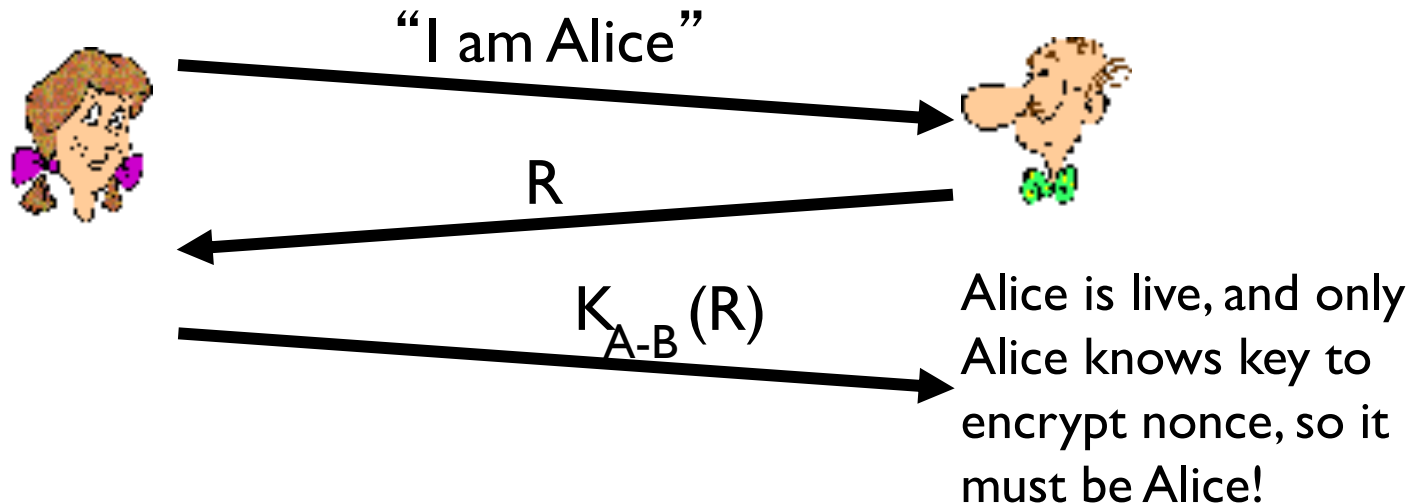


- ❖ receiver extracts IV
- ❖ inputs IV, shared secret key into pseudo random generator, gets keystream
- ❖ XORs keystream with encrypted data to decrypt data + ICV
- ❖ verifies integrity of data with ICV
 - note: message integrity approach used here is different from MAC (message authentication code) and signatures (using PKI).

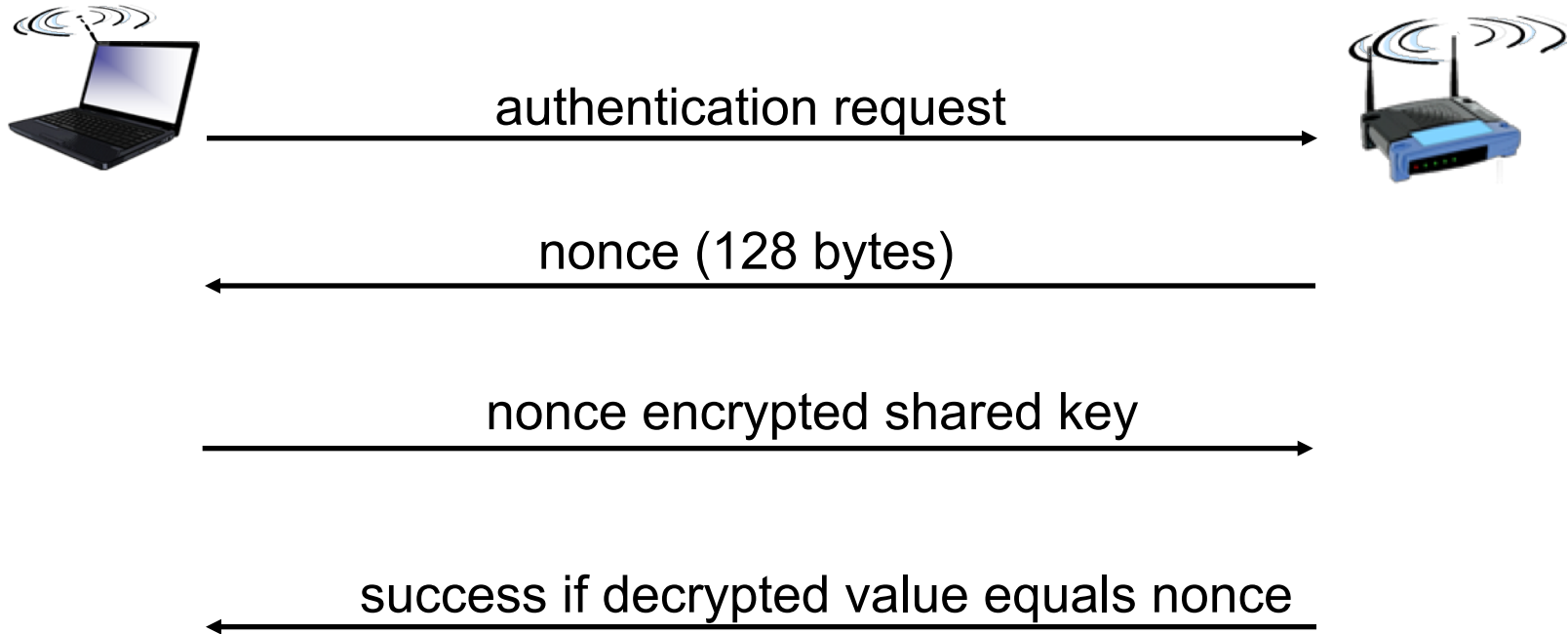
End-point authentication w/ nonce

Nonce: number (R) used only *once* –*in-a-lifetime*

How to prove Alice “live”: Bob sends Alice **nonce**, R. Alice must return R, encrypted with shared secret key



WEP authentication



Notes:

- ❖ not all APs do it, even if WEP is being used
- ❖ AP indicates if authentication is necessary in beacon frame
- ❖ done before association

Breaking 802.11 WEP encryption

security hole:

- ❖ 24-bit IV, one IV per frame, -> IV's eventually reused
- ❖ IV transmitted in plaintext -> IV reuse detected

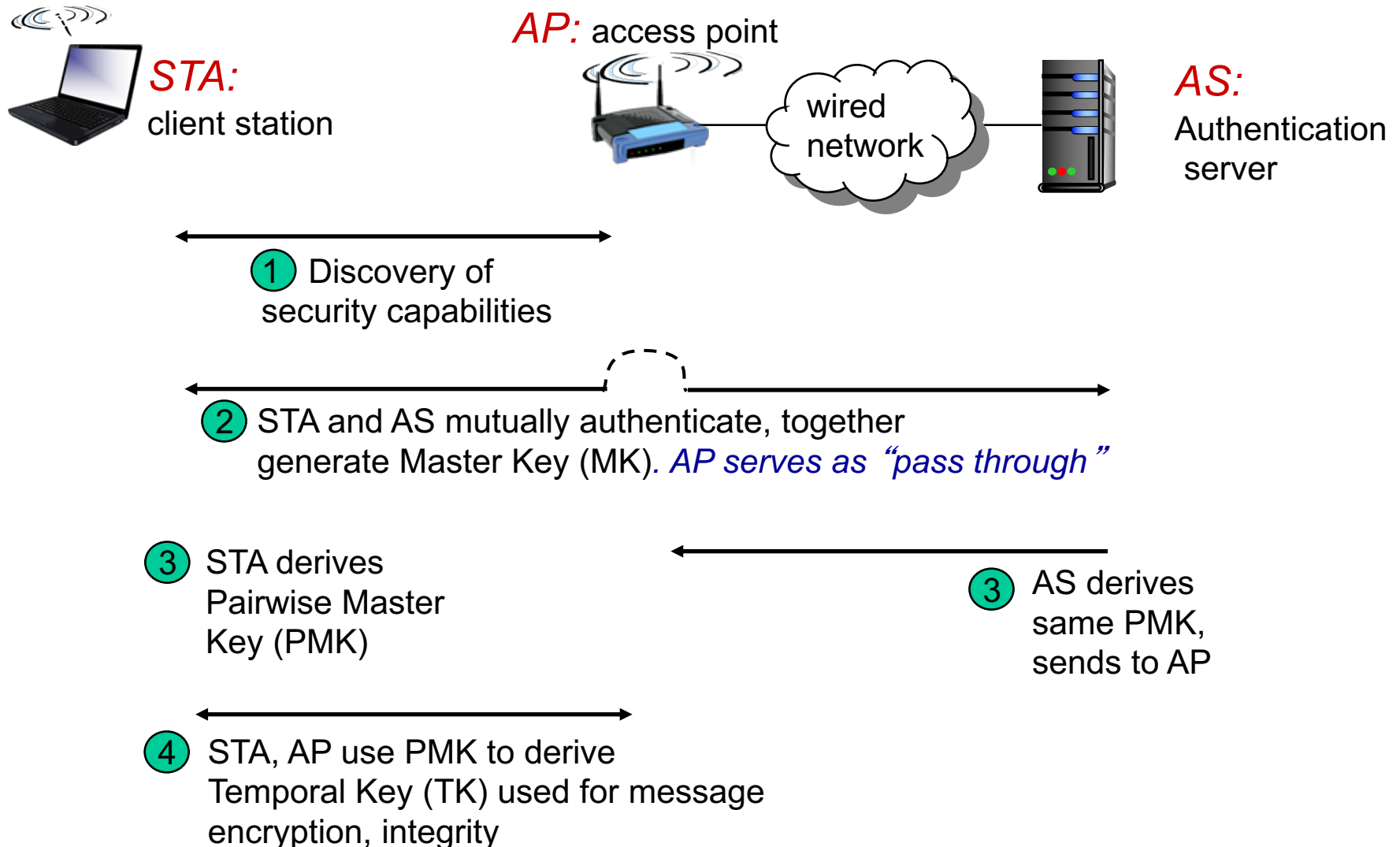
attack:

- Trudy causes Alice to encrypt known plaintext $d_1 d_2 d_3 d_4 \dots$
- Trudy sees: $c_i = d_i \text{ XOR } k_i^{\text{IV}}$
- Trudy knows $c_i d_i$, so can compute k_i^{IV}
- Trudy knows encrypting key sequence $k_1^{\text{IV}} k_2^{\text{IV}} k_3^{\text{IV}} \dots$
- Next time IV is used, Trudy can decrypt!

802.11i: improved security

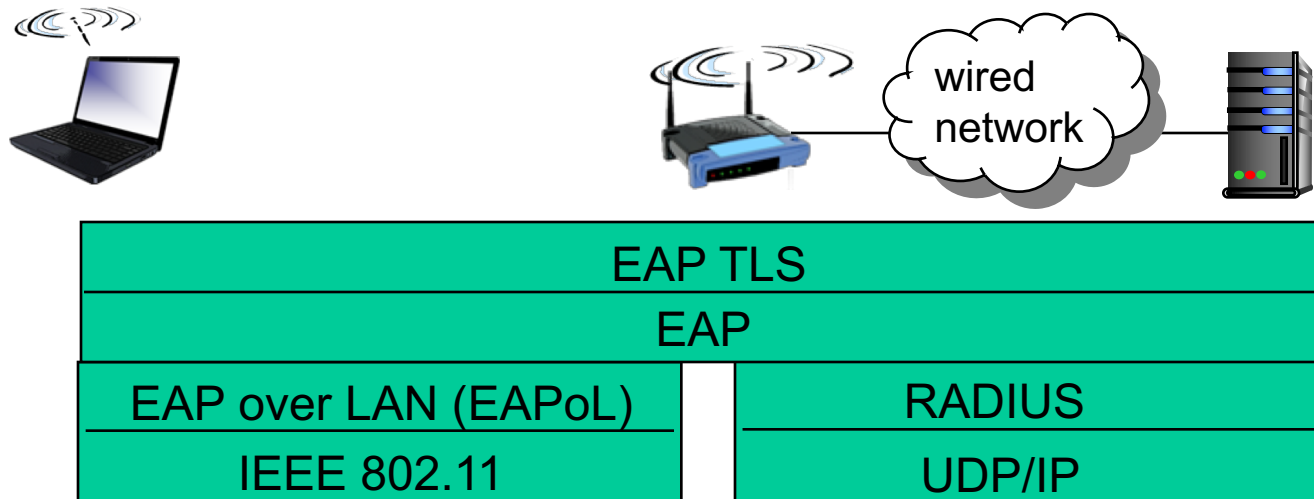
- ❖ numerous (stronger) forms of encryption possible
- ❖ provides key distribution
- ❖ uses authentication server separate from access point

802.11i: four phases of operation



EAP: extensible authentication protocol

- ❖ EAP: end-end client (mobile) to authentication server protocol
- ❖ EAP sent over separate “links”
 - mobile-to-AP (EAP over LAN)
 - AP to authentication server (RADIUS over UDP)



Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

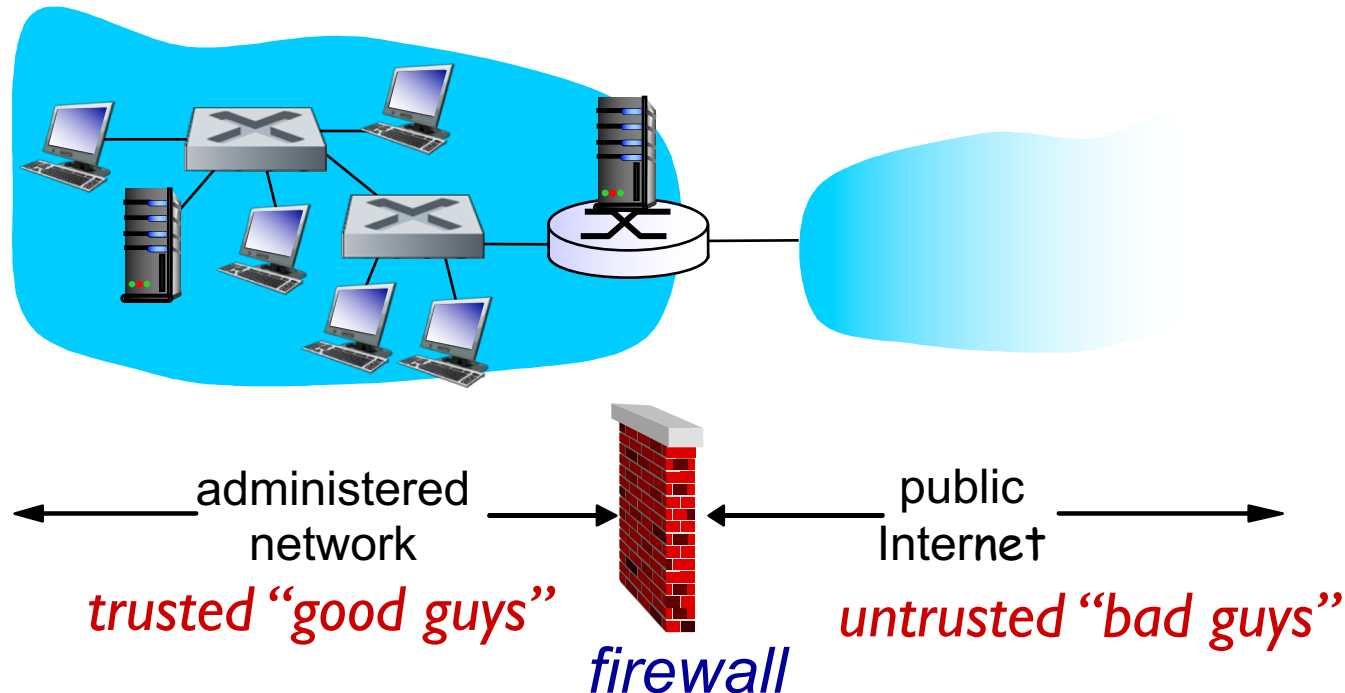
8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Firewalls

firewall

isolates organization's internal net from larger Internet, allowing some packets to pass, blocking others



Firewalls: why

prevent denial of service attacks:

- ❖ SYN flooding: attacker establishes many bogus TCP connections, no resources left for “real” connections

prevent illegal modification/access of internal data

- ❖ e.g., attacker replaces CIA’s homepage with something else

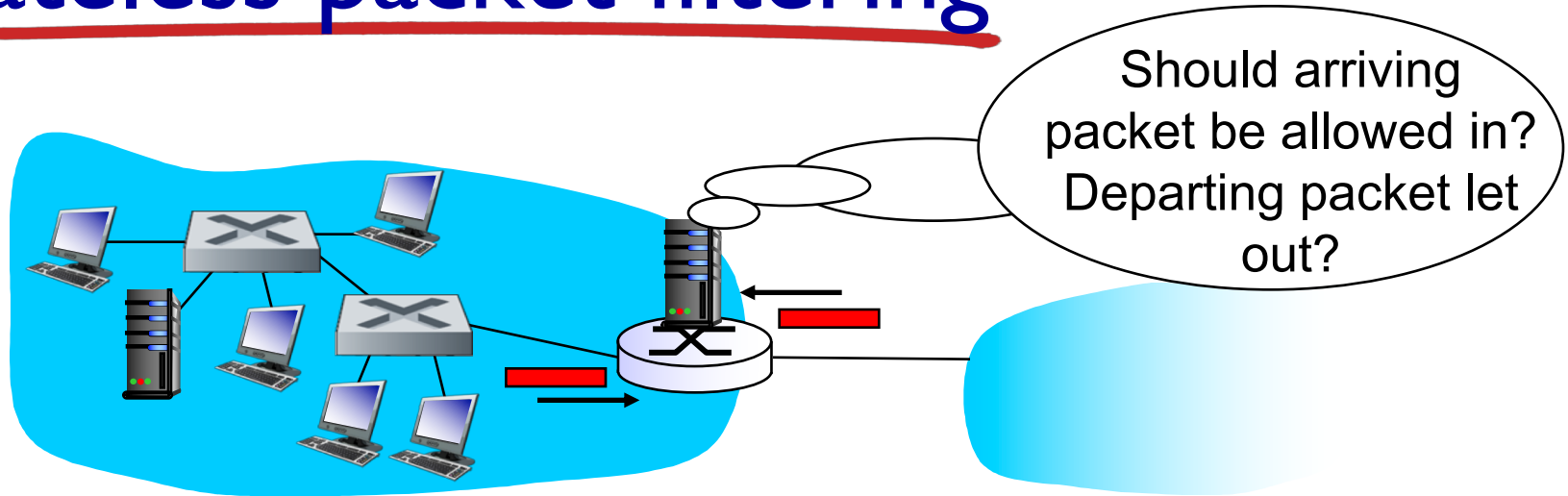
allow only authorized access to inside network

- ❖ set of authenticated users/hosts

three types of firewalls:

- ❖ stateless packet filters
- ❖ stateful packet filters
- ❖ application gateways

Stateless packet filtering



- ❖ internal network connected to Internet via *router firewall*
- ❖ router *filters packet-by-packet*, decision to forward/drop packet based on:
 - source IP address, destination IP address
 - TCP/UDP source and destination port numbers
 - ICMP message type
 - TCP SYN and ACK bits

Stateless packet filtering: example

- ❖ *example 1:* block incoming and outgoing datagrams with IP protocol field = 17 and with either source or dest port = 23
 - *result:* all incoming, outgoing UDP flows and telnet connections are blocked
- ❖ *example 2:* block inbound TCP segments with ACK=0.
 - *result:* prevents external clients from making TCP connections with internal clients, but allows internal clients to connect to outside.

Stateless packet filtering: more examples

<i>Policy</i>	<i>Firewall Setting</i>
No outside Web access.	Drop all outgoing packets to any IP address, port 80
No incoming TCP connections, except those for institution's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets - except DNS and router broadcasts.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being tracerouted	Drop all outgoing ICMP TTL expired traffic

Access Control Lists

❖ **ACL:** table of rules, applied top to bottom to incoming packets: (action, condition) pairs

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

Stateful packet filtering

❖ *stateless packet filter*: heavy handed tool

- admits packets that “make no sense,” e.g., dest port = 80, ACK bit set, even though no TCP connection established:

action	source address	dest address	protocol	source port	dest port	flag bit
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK

❖ *stateful packet filter*: track status of every TCP connection

- track connection setup (SYN), teardown (FIN): determine whether incoming, outgoing packets “makes sense”
- timeout inactive connections at firewall: no longer admit packets

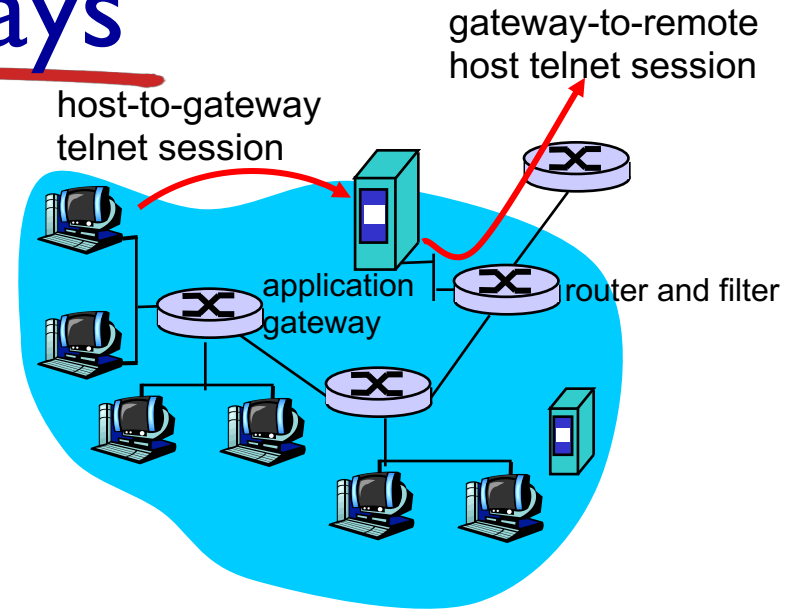
Stateful packet filtering

- ❖ ACL augmented to indicate need to check connection state table before admitting packet

action	source address	dest address	proto	source port	dest port	flag bit	check conxion
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----	X
deny	all	all	all	all	all	all	

Application gateways

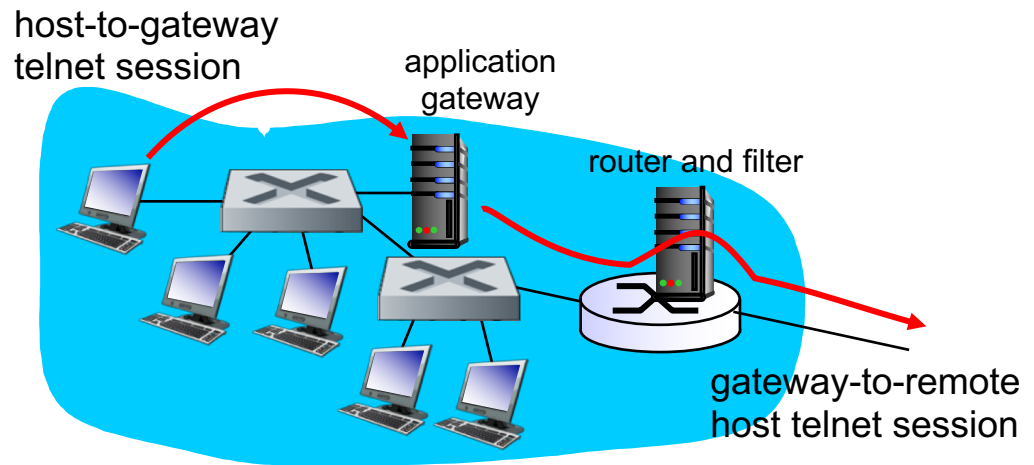
- ❖ filters packets on application data as well as on IP/TCP/UDP fields.
- ❖ *example:* allow select internal users to telnet outside.



1. require all telnet users to telnet through gateway.
2. for authorized users, gateway sets up telnet connection to dest host. Gateway relays data between 2 connections
3. router filter blocks all telnet connections not originating from gateway.

Application gateways

- ❖ filter packets on application data as well as on IP/TCP/UDP fields.
- ❖ *example:* allow select internal users to telnet outside



1. require all telnet users to telnet through gateway.
2. for authorized users, gateway sets up telnet connection to dest host. Gateway relays data between 2 connections
3. router filter blocks all telnet connections not originating from gateway.

Limitations of firewalls, gateways

- ❖ *IP spoofing*: router can't know if data “really” comes from claimed source
- ❖ if multiple app's. need special treatment, each has own app. gateway
- ❖ client software must know how to contact gateway.
 - e.g., must set IP address of proxy in Web browser
- ❖ filters often use all or nothing policy for UDP
- ❖ *tradeoff*: degree of communication with outside world, level of security
- ❖ many highly protected sites still suffer from attacks

Intrusion detection systems

❖ packet filtering:

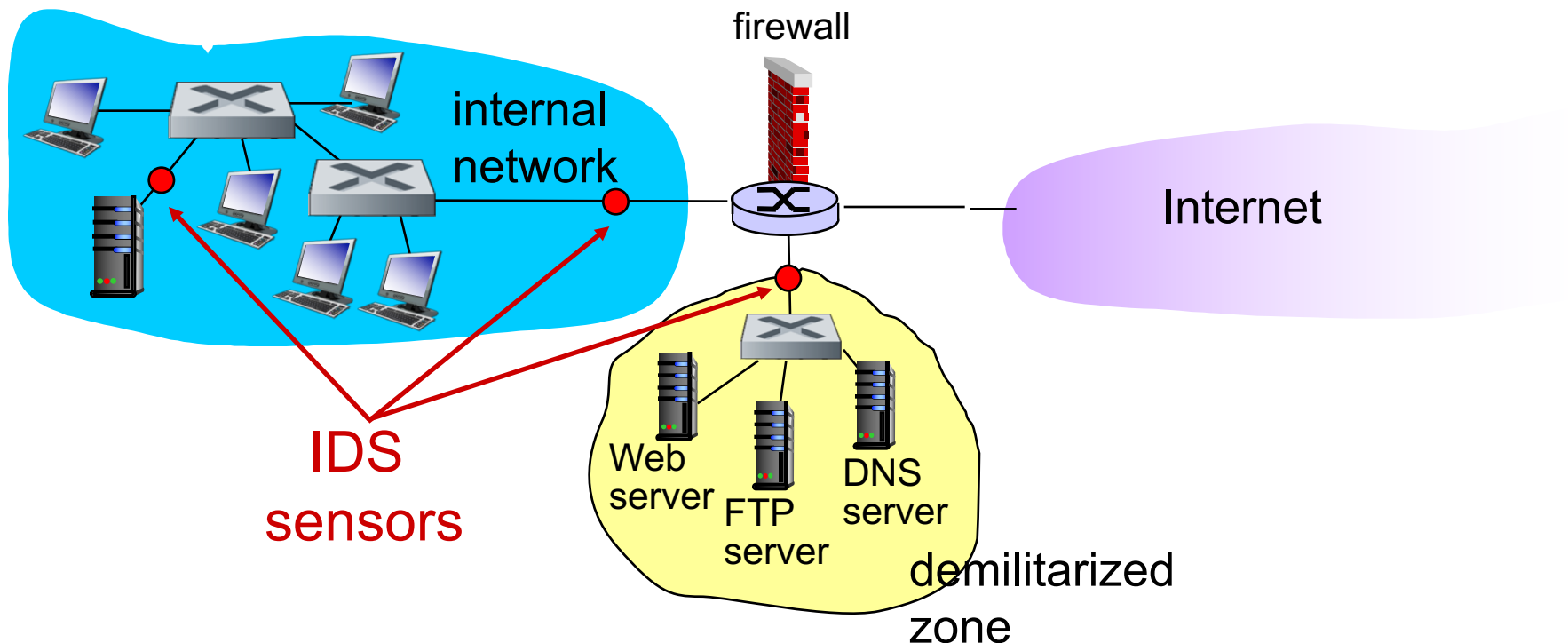
- operates on TCP/IP headers only
- no correlation check among sessions

❖ *IDS: intrusion detection system*

- *deep packet inspection*: look at packet contents (e.g., check character strings in packet against database of known virus, attack strings)
- *examine correlation* among multiple packets
 - port scanning
 - network mapping
 - DoS attack

Intrusion detection systems

- ❖ multiple IDSs: different types of checking at different locations



Network Security (summary)

basic techniques.....

- cryptography (symmetric and public)
- message integrity
- end-point authentication

.... used in many different security scenarios

- secure email
- secure transport (SSL)
- IP sec
- 802.11

operational security: firewalls and IDS